

Data Engineer — Rapid-Fire Mock Drill

56 questions with one-line answers — practice out loud

fundamentals, Spark/Delta, AWS, security, modeling, lakes, Oracle, standards

Practice **out loud**: cover the answer, say yours, compare. One-line answers on purpose — in a real interview you'd expand with an example. Covers all three JDs (security cloud DE, Databricks/ Delta, Oracle/Iceberg lakehouse).

A. Fundamentals & pipelines

1. **ETL vs ELT?** ETL transforms before load; ELT loads raw then transforms in the warehouse/lake (modern default).
2. **Batch vs streaming?** Batch = scheduled bulk; streaming = continuous events (Kinesis/Kafka) for near-real-time.
3. **What is CDC?** Capture inserts/updates/deletes from a source DB (DMS) with low impact — fresher than dumps.
4. **Incremental load?** Process only new/changed data via watermark/CDC/bookmarks; upsert with MERGE; keep it idempotent.
5. **Idempotent pipeline?** Re-running gives the same result — overwrite-by-partition or MERGE on a key.
6. **What is IaC?** Infra defined as code (Terraform/CloudFormation) — repeatable, reviewable, version-controlled.
7. **How do you orchestrate?** Airflow/Step Functions/Databricks Workflows — DAGs with retries, alerts, event triggers.
8. **Why Parquet?** Columnar + compressed + typed → scan less, faster, cheaper.
9. **Partitioning?** Split by a filter column (usually date) so queries prune to relevant folders.
10. **Small-files problem?** Too many tiny files hurt performance → compact/coalesce to ~128MB+.

B. Spark / Databricks / Delta

1. **Transformation vs action?** Transformations are lazy (build the DAG); actions trigger execution.
2. **What's a shuffle?** Data moved across the network for join/groupBy/distinct — the main cost.
3. **repartition vs coalesce?** repartition = full shuffle (up/down); coalesce = no shuffle, reduce only.
4. **Broadcast join?** Send a small table to every executor to avoid shuffling the big one.
5. **Data skew fix?** Salting, AQE skew handling, or isolate the hot key.
6. **Why avoid UDFs?** Opaque to Catalyst + Python serialization overhead; prefer built-ins/pandas UDFs.
7. **What is Delta Lake?** ACID table format on Parquet: MERGE/upserts, time travel, schema evolution.
8. **OPTIMIZE / Z-ORDER?** Compact small files; co-locate data on a column for skipping.
9. **Medallion architecture?** Bronze (raw) → Silver (clean) → Gold (aggregated).
10. **Unity Catalog?** Databricks centralized governance — fine-grained access + lineage.

C. AWS & data stores

1. **S3, Glue, Lambda roles?** S3 = lake storage; Glue = serverless Spark ETL + catalog; Lambda = event-driven compute.
2. **Athena vs Redshift?** Athena = serverless ad-hoc SQL on S3; Redshift = warehouse for frequent, high-concurrency BI.
3. **Redshift DISTKEY vs SORTKEY?** DISTKEY = co-locate joins; SORTKEY (date) = skip blocks on filters.
4. **How to load Redshift fast?** COPY from many split Parquet files in parallel, same region.
5. **Glue Data Catalog?** Shared Hive metastore used by Athena, Redshift Spectrum, EMR, Glue.
6. **Control AWS cost?** Parquet + partitioning (scan less), pause/Serverless, lifecycle to Glacier, compact files.

D. Security (the differentiator)

1. **Encryption at rest / in transit?** KMS (SSE-KMS, encrypted DBs) at rest; TLS/HTTPS + SSL in transit.
2. **Least privilege?** Narrow IAM roles/policies, per-service roles, no wildcards, temporary creds (STS).
3. **Secrets management?** Secrets Manager / SSM (KMS-encrypted), fetched at runtime, rotated — never in code.
4. **Protect PII?** Classify + mask/tokenize, column/row-level security, encrypt, Macie to discover, CloudTrail audit.
5. **Prevent SQL injection?** Parameterized queries / bind variables — never concatenate user input into SQL.
6. **Secure coding basics?** Validate input, no secrets in code, dependency scanning, safe error handling, least privilege.
7. **Network security?** VPC, private subnets for data, security groups, VPC endpoints, block public S3.

E. Data modeling

1. **Dimensional vs normalized?** Star (fact + denormalized dims) for analytics; 3NF for OLTP/integration.
2. **Fact vs dimension?** Fact = measures/events; dimension = descriptive context (customer/product/date).
3. **SCD Type 2?** Track history with effective/expiry dates + current flag (new row per change).
4. **Surrogate key?** System-generated key for dim rows — stable joins, supports SCD2.
5. **Star vs snowflake?** Star = denormalized dims (fast); snowflake = normalized dims (more joins).

F. Data lakes / Iceberg / lakehouse

1. **Data lake vs warehouse vs lakehouse?** Lake = cheap any-format storage; warehouse = fast SQL; lakehouse = warehouse features on the lake.
2. **How to update/delete in a lake?** Table formats — Iceberg/Delta/Hudi (ACID, upserts, time travel).
3. **What is Apache Iceberg?** Open table format on object storage: ACID, schema/partition evolution, time travel, engine-neutral.
4. **Iceberg vs Delta vs Hudi?** All ACID/upsert/time-travel; Iceberg = engine-neutral; Delta = Databricks; Hudi = streaming upserts.
5. **Govern a lake?** Lake Formation/Unity Catalog — table/column/row grants, catalog as source of truth, audit.

G. Oracle & ops (JD 3)

1. **Ingest flat files to Oracle?** External tables, SQL*Loader (direct-path), partition exchange for fast publish.
2. **External table?** Table over a file/object store Oracle reads on demand — no pre-load.
3. **Oracle query tuning?** Partitioning + pruning, indexes (bitmap for DW), DBMS_STATS, read plans, parallel, materialized views.
4. **Materialized view?** Pre-computed (aggregate) results with query rewrite — speeds repeated heavy queries.
5. **Cloud-to-Oracle integration?** External tables over object storage / DBMS_CLOUD to read Parquet/files.
6. **Shell orchestration?** Bash + cron running load/SQL steps with exit-code checks, logging, restartability.
7. **Linux perf tuning?** top/vmstat/iostat/df to find CPU/mem/IO bottlenecks; tune limits, disk, concurrency.

H. Standards & soft skills

1. **CI/CD for data?** Git + PR reviews + tests in CI + deploy via Terraform/Databricks bundles across envs.
 2. **Data quality?** dbt/GE/DLT tests (null/unique/range/freshness) that fail the pipeline on bad data.
 3. **How do you document?** README/design docs, docstrings, lineage/catalog, runbooks.
 4. **Explain a pipeline to a non-tech stakeholder?** Analogy + business outcome, no jargon.
 5. **Handle a prod incident?** Scope → logs/metrics → isolate → fix → add a preventive check → communicate.
 6. **What is MDM / golden record?** Match+merge records into one trusted version of a core entity (customer/product).
-

How to drill

- Day 1–2: read aloud, uncover answers.
- Day 3: have someone quiz you at random.
- Before the interview: skim only the sections matching **that company's JD**.
- Always end technical answers with the **why** (performance, cost, reliability, security).

